SEES Project - Web Wii

Mika Baëza, Mohammad Parto, Kawtar Bidoukhach

April 2026

Contents

1 Introduction

In 2006, Nintendo introduced its new console to the public: the Nintendo Wii. At the time, this game console was a breakthrough due to the specificity of its controller, the *Wii Remote*. The particularity of this remote is its ability to act as an intuitive pointing device. The Wii Remote can be seen as a laser pointer, except that the position it points to on screen becomes a cursor, allowing users to interact with on-screen elements in a natural and intuitive way.

In this project, we aimed to recreate this intuitive device using modern embedded hardware. The system consists of one Adafruit Metro ESP32-S3 boards communicating over Bluetooth Low Energy (BLE) to a webpage opened on a PC.

This report describes the system architecture, hardware design, sensor modelling, firmware implementation, and experimental results of this project.

2 Background

2.1 The Original Wii Remote

The Nintendo Wii Remote achieves its pointing and motion-sensing capabilities through two complementary subsystems: an infrared camera and an accelerometer.

IR-based pointing. The *sensor bar* bundled with the Wii console is entirely passive: it contains a pair of infrared LEDs at each end of a $\sim$20 cm bar and transmits no data [3]. All sensing occurs inside the remote, which contains a forward-facing IR camera. This camera detects up to four IR sources and outputs their 2D image-plane coordinates directly [3]. From the known physical separation of the two LED clusters, the console software infers where the remote is pointing on screen. The sensor bar is nothing more than a pair of fixed IR beacons: any two IR LEDs at a known separation can serve as a functional substitute.

Accelerometer. The original Wii Remote included a 3-axis accelerometer for gesture and tilt sensing, but lacked independent rotational tracking. This was addressed in 2009 with the Wii MotionPlus accessory, which added a multi-axis MEMS gyroscope [1].

Relevance to this project. This project mostly mirrors the same architecture: two fixed IR LEDs replace the sensor bar, and the remote's IR camera tracks them to derive cursor coordinates. Anyway, for this project, we decided to use a IMU that combines accelerometer and gyroscope data to give a ready-to-use orientation data, since our goal is to just have a pointing device, and not detect any shaking or other user movement. Thereofre having directly the orientation of the remote makes the computation of the cursor position easier.

2.2 Bluetooth Low Energy (BLE)

BLE organises data exchange around the *Generic Attribute Profile* (GATT) model. A *peripheral* device exposes one or more *services*, each containing one or more *characteristics*. A *central* device (here, the browser running on the PC) can read, write, or subscribe to those characteristics. In this project the remote acts as the peripheral, advertising a single service with one characteristic that carries the cursor packet.

Two transfer modes are available for characteristics. In *polling*, the central explicitly reads the characteristic at whatever rate it chooses. With *notifications*, the peripheral pushes an updated value to all subscribed centrals as soon as it is ready. Notifications were preferred here: they invert the flow of control so the remote drives the update rate, and deliver data at the sensor cadence without requiring the central to time its reads precisely, allowing a future potential implementation of a *sleep mode* (not implemented in the scope of this project).

BLE was chosen over Wi-Fi for two practical reasons. First, BLE introduces lower and more predictable per-packet latency, which is important for a real-time pointing device. Second, BLE consumes significantly less power than Wi-Fi, a relevant constraint for a battery-powered handheld unit.

2.3 Inertial Measurement Units and Sensor Fusion

An *Inertial Measurement Unit* (IMU) is a device that measures the forces and rotational rates acting on a rigid body. A typical IMU combines an *accelerometer*, which measures linear acceleration along three axes, a *gyroscope*, which measures angular velocity around three axes, and optionally a *magnetometer*, which measures the local magnetic field to provide an absolute heading reference.

Raw data from each sensor is individually imperfect: accelerometers are noisy and sensitive to vibration, while gyroscopes suffer from drift as integration errors accumulate over time. *Sensor fusion* combines both sources to produce an orientation estimate that is more accurate and stable than either alone.

The BNO085 used in this project integrates all three sensor types on a single chip and runs its own fusion algorithm in dedicated hardware [6]. It exposes the final orientation directly as Euler angles (yaw, pitch, roll) at 100 Hz. This removes the need to implement a fusion filter manually and simplifies the firmware to a single serial read per cycle.

2.4 IR-Based Pointing and Perspective Projection

The pointing subsystem relies on a passive reference bar containing two IR LEDs separated by a fixed distance of 200 mm, placed on top of the screen. The remote's onboard IR camera (DFRobot SEN0158) detects up to four IR sources and reports their pixel coordinates over I2C [7]. In normal operation, only the two reference LEDs are visible, so only the first two reported points are used.

The camera operates at a resolution of 1024×768 px with a horizontal field of view of 33 deg and a vertical field of view of 23 deg. Because in our case the camera is mounted sideways in the remote, its width and height axes are physically swapped relative to the world orientation; the firmware accounts for this by exchanging the W and H parameters, and the x and y values of any sensor read (see after).

The 2-D pixel observations are converted into a 3-D cursor position through a perspective back-projection, detailed in Annex A. Each observed pixel (u_i, v_i) defines a ray direction $\mathbf{d}_i$ in the camera frame, which is rotated into the global frame using the IMU orientation. The 3-D position of the remote is then recovered by finding the point best consistent with both observed pixels. The 2-D cursor coordinates are finally obtained by intersecting the remote's pointing ray with the screen plane, defined as $Y = 0$ in the global frame.

3 Description

3.1 System Architecture

The system is composed of one ESP32-S3 boards exclusively over BLE with a web page, and a reference device placed on top of the screen.

Remote board The handheld unit is battery-powered and hosts a BNO085 IMU (UART) for orientation sensing, an IR Camera which tracks the IR LEDs on the reference device to extract cursor coordinates, and several buttons. It acts as a BLE *peripheral* (forwarding data to the PC) at maximum 100 Hz.

Reference device The reference device is USB-powered and hosts two IR leds separated of 20cm, It does not contain any internal logic and is just used as a reference line for the remote IR camera. Figure 1 shows the overall system block diagram.

Web page The user interface is implemented as a web page built with Svelte and SvelteKit. Communication with the remote is handled through the Web Bluetooth API [4], supported natively in Google Chrome since version 56, which allows a web page to connect directly to a BLE peripheral without any native application or driver.

Figure 1 shows the overall system block diagram.

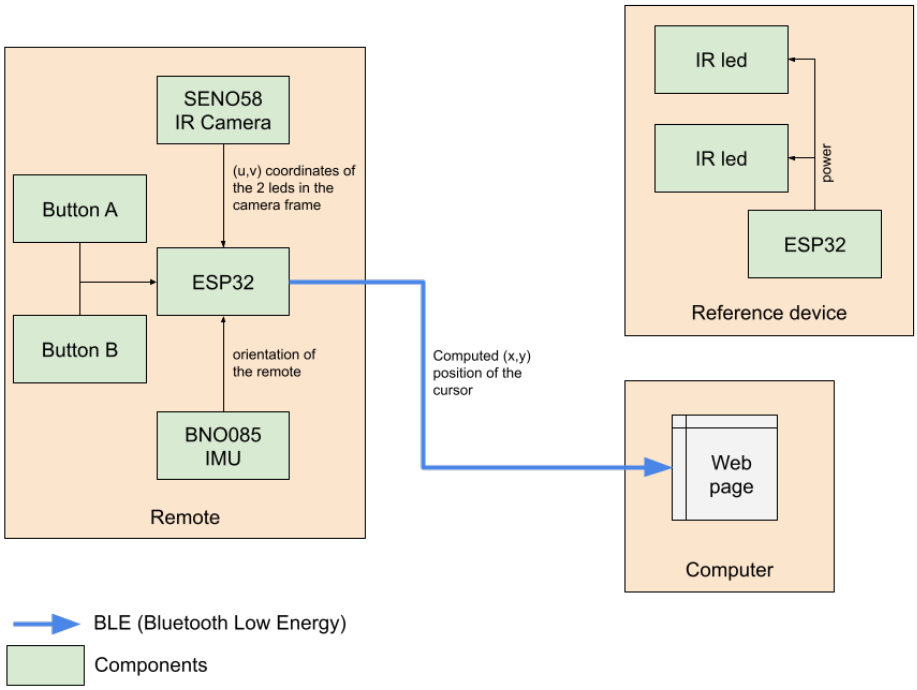

Figure 1: System architecture: Remote $\xrightarrow{\text{BLE}}$ PC

3.2 Cursor Estimation

The goal of cursor estimation is to recover the 3-D position $\mathbf{P} = (p_x, p_y, p_z)^\top$ of the remote in the global frame from two pieces of information: the IMU orientation and the pixel coordinates of the two IR beacons observed by the on-board camera.

The IR camera back-projects each observed pixel (u_i, v_i) into a ray direction $\mathbf{d}_i$ in the camera frame using the known camera intrinsics (focal lengths f_x, f_y derived from the sensor's field of view). Each ray is then rotated into the global frame using the rotation matrix $\mathbf{R}$ built from the IMU's yaw, pitch and roll:

$$\mathbf{r}_i = \mathbf{R}\,\mathbf{d}_i, \qquad i = 1, 2. \tag{1}$$

Since each beacon L_i must lie along its corresponding ray, the remote's position satisfies $\mathbf{P} = L_i - \lambda_i\,\mathbf{r}_i$ for some scalar depth $\lambda_i > 0$. Solving the equation of the two expressions for $\mathbf{P}$ leads to an overdetermined 3×2 linear system in (λ_1, λ_2), which is solved in closed form by least squares. The final position estimate is:

$$\mathbf{P} = \frac{1}{2}\Big[(L_1 - \lambda_1\,\mathbf{r}_1) + (L_2 - \lambda_2\,\mathbf{r}_2)\Big]. \tag{2}$$

The system is well-conditioned as long as the two beacons do not project onto the same pixel, which is guaranteed whenever the reference bar is visible. The full derivation, including the explicit closed-form expressions for λ_1 and λ_2, is given in Annex A.

3.3 BLE Communication Protocol

The remote transmits a 14-byte packet to the console at ∼100 Hz:

```
struct RemotePacket {
    int32_t x;           // mm * 100        (4 bytes)
    int32_t y;           // mm * 100        (4 bytes)
    int32_t pitch;       // degrees * 100   (4 bytes)
    uint8_t buttonMask;  // bitmask         (1 byte)
    uint8_t missMask;    // bitmask         (1 byte)
};                       // Total: 14 bytes
```

Listing 1: BLE packet structure (remote → PC)

3.4 Firmware Design

The remote firmware was written in C++ with Arduino IDE. It follows a straightforward loop: read the IMU, read the IR camera, read the buttons, compute the cursor position, and push a BLE notification (all within a 10 ms window to sustain the 100 Hz target rate). Each hardware peripheral is encapsulated in its own initialisation and read function, keeping the main loop compact.

BNO085 initialisation and reading. The BNO085 is operated in its UART RVC (Raw Vector Continuous) mode. Although the sensor exposes an I2C interface, its datasheet notes that it is not compatible with the I2C specification [6], which caused compatibility issues with the ESP32-S3. The UART protocol was used instead. More specificatlly UART-RVC was preferred because this the board only needs to *receive* orientation data from the sensor (nothing needs to be sent back). UART RVC is designed precisely for this case, reducing the firmware interaction to a single read per cycle. The `Adafruit_BNO08x_RVC` library handles frame synchronisation and parsing over a dedicated hardware UART (Serial1, 115200 baud, pins 4 and 5).

In consequence, because the BNO085 transmits frames continuously (and asynchronously), a read attempt may catch the UART buffer mid-frame. If the first `rvc.read()` call fails, the firmware waits 3 ms, and retries once before declaring the reading as missed.

A calibration offset is maintained for each axis. On every read, the raw Euler angles output by the sensor are corrected by subtracted by the offset, so that the resting orientation of the remote reads as $(0, 0, 0)$ (pointing toward the center of the screen). The offset is updated whenever the user presses button B, capturing the current orientation as the new zero reference. Additionally, the firmware forces ten automatic resets during the first ten loop cycles at power-on, so the remote self-calibrates to whatever orientation it happens to be held in at start-up (ideally pointing to the center of the screen).

IR camera initialisation and reading. The DFRobot SEN0158 IR positioning camera is connected over I^2C at address `0x58` (shifted from the raw sensor address `0xB0`). On the board the camera is connected to the sefault SDA and SCL port. Initialisation consists of writing a fixed six-register sequence that configures the camera's sensitivity and output format, as specified in the datasheet [7].

Each read requests 16 bytes from register `0x36`. The camera encodes the pixel coordinates of up to four detected IR sources across those bytes, with two upper bits of each coordinate packed into a shared status byte. A coordinate value of 1023 signals that the corresponding source was not detected; if either of the first two spots reports this default value, the read is declared as missed and the firmware retains the last valid position.

Because the camera is mounted sideways in the remote enclosure, its reported x and $y$axes are physically transposed relative to the world frame. The firmware corrects this by swapping the pixel components before writing them into the sensor payload.

Button reading and encoding. Two push-buttons are connected to GPIO pins 7 and 9 and configured as digital inputs. Button A (pin 7) is the action button: its state is read each cycle and encoded into bit 0 of the `buttonMask` byte included in the BLE payload. Button B (pin 9) is a reset button that triggers a gyroscope origin reset when held; it is not forwarded to the host.

BLE peripheral setup. The ESP32 is configured as a BLE peripheral under the device name `WiiRemote`. A single GATT service is created with one characteristic that carries the cursor payload; the characteristic is declared with the `READ`, `WRITE` and `NOTIFY` properties, enabling the web client to subscribe to updates. Two custom 128-bit UUIDs identify the service and characteristic, ensuring the browser-side Web Bluetooth connection request can uniquely target the remote.

A server callback handles connection events. On connect, the `computerConnected` flag is set and the status LED turns white. On disconnect, the flag is cleared and BLE advertising is restarted after a 1 s debounce delay, making the remote immediately discoverable again without requiring a reboot.

Cursor computation. Once the sensor readings are available, the firmware calls `localise_remote()`, which converts the IMU Euler angles (degrees to radians) and the two IR pixel pairs into a 3-D position vector and a pointing-direction ray in the global frame, following the algorithm described in Annex A. If the computation fails (either because the two rays are nearly parallel ($\delta < 10^{-9}$) or because one of the recovered depths λ_i is negative) the function returns `false` and bit 2 of the miss mask is set.

Remark 1 (Axis assignment in the firmware) *The physical mounting of the BNO085 on the remote imposes an axis assignment that differs from the convention defined in Annex A: Y serves as the depth axis where the appendix uses Z, and Z is the vertical axis where the appendix uses Y. The localisation algorithm is otherwise identical; only the axis labels differ.*

The 3-D position is then projected onto the screen plane ($Y = 0$ in the global frame) by ray casting: the parameter $t = -p_y/r_y$ gives the intersection, and the cursor coordinates follow as $(p_x + t\,r_x,\ p_z + t\,r_z)$. Both coordinates and the pitch angle are scaled by a factor of 100 and stored as `int32_t`, providing two decimal places of precision while avoiding floating-point in the BLE payload.

Main loop and timing. The loop is driven by a `millis()` timer that fires every `NOTIFY_INTERVAL_MS` $= 10$ ms ($\approx$100 Hz). Within each tick, the firmware executes the

pipeline in order: read the gyroscope, optionally reset the gyroscope origin, read the IR camera, update the board state, assemble the cursor payload, and push a BLE notification.

Failures at each stage are tracked through a three-bit `MISS_MASK`: bit 0 signals a missed IMU frame, bit 1 a missing IR beacon, and bit 2 a failed localisation. The mask is included in every BLE packet so the host can distinguish invalid cursor computations from valid ones. The onboard NeoPixel LED also reflects the mask: it turns off for any non-zero value and returns to its normal colour when all three stages succeed. A debug dump of all intermediate values is printed to the serial monitor once per second.

4 Experimental Results and Conclusion

4.1 Limitations and Trade-offs

IR LED emission angle. The IR LEDs used on the reference device have a narrow emission cone of approximately 20°. When the remote is pointed too far off-axis relative to the reference bar, the IR camera loses track of one or both beacons, causing cursor tracking to fail. This is not a fundamental limitation of the pointing algorithm but rather a consequence of the specific LEDs chosen. Replacing our current LEDS with wider-angle IR emitters would directly extend the usable pointing range without any software change.

BNO085 sensor fusion inertia. The BNO085 onboard fusion algorithm prioritises orientation stability over instantaneous responsiveness. In practice, this introduces a slight lag on fast rotational movements, making rapid gestures feel softly delayed on screen.

Browser compatibility. The Web Bluetooth API is currently only supported in Chromium-based browsers [4], restricting the system to Chrome, Edge or Brave etc... on desktop. Safari and Firefox do not implement the API, which limits the portability of the web interface.

4.2 Conclusion

This project successfully reproduced the core experience of the Nintendo Wii Remote using modern embedded hardware. The full system pipeline — IR-based cursor pointing, absolute orientation sensing, button input, BLE transmission, and web-based rendering demonstrated that such an interface can be built with modern components and a browser as the only software dependency for the user.

Two practical limitations were identified during use. First, the IR LEDs used on the reference device have a narrow $\sim$20° emission cone. This is an inherent constraint of the chosen LEDs rather than of the pointing algorithm, and could be addressed by substituting wider-angle IR emitters. Second, the BNO085's onboard sensor fusion introduces a slight inertia on fast rotational movements, causing a perceptible lag between physical gesture and on-screen response. This is a known trade-off of gyroscope-based fusion filters, which prioritise stability over instantaneous responsiveness.

Overall, the system met the objectives of the project. The design validates the use of a passive IR reference, an onboard fusion IMU, and the Web Bluetooth API as a viable and

accessible stack for building motion-controlled pointing devices, with no native software required on the host machine.

AI Disclosure

This document was written by the authors. Generative AI tools were used to assist with the formulation of certain passages and the LaTeX typesetting. All content and results were reviewed and validated by the authors.

5 References

References

[1] Wikipedia, *Wii MotionPlus*. https://en.wikipedia.org/wiki/Wii_MotionPlus

[2] Nintendo, *Iwata Asks: Wii MotionPlus — Combining Two Sensors*, 2009. https://www.nintendo.com/en-gb/Iwata-Asks/Iwata-Asks-Wii-MotionPlus/Read-more/2-Combining-Two-Sensors/2-Combining-Two-Sensors-225646.html

[3] Wiibrew, *Wiimote wiki*. https://wiibrew.org/wiki/Wiimote

[4] F. Beaufort, *Communicating with Bluetooth Devices over JavaScript*, Google Chrome for Developers, 2015. https://developer.chrome.com/docs/capabilities/bluetooth

[5] MDN Web Docs, *Web Bluetooth API*. https://developer.mozilla.org/en-US/docs/Web/API/Web_Bluetooth_API

[6] Adafruit Industries, *BNO085 Datasheet*, 2025. https://cdn-learn.adafruit.com/downloads/pdf/adafruit-9-dof-orientation-imu-fusion-breakout-bno085.pdf

[7] DFRobot, *Gravity: IR Positioning Camera Datasheet*, 2025. https://mm.digikey.com/Volume0/opasdata/d220001/medias/docus/2148/Positioning_IR_Camera_Web.pdf

A 3-D Localisation of the Remote

A.1 Problem Statement

A.1.1 Goal

Recover the 3-D position $\mathbf{P} = (p_x,\, p_y,\, p_z)^\top \in \mathbb{R}^3$ of a Wii remote in a fixed global frame $\mathcal{O}$, given:

- the orientation of the remote expressed as Euler angles (yaw ψ, pitch θ, roll ϕ);
- the pixel coordinates of two infrared beacons captured by the remote's on-board camera;
- the intrinsic parameters of that camera.

A.1.2 Known quantities

- Two beacons fixed in the global frame: $L_1 = (d,\, 0,\, 0)^\top$ and $L_2 = (-d,\, 0,\, 0)^\top$, separated by a known distance $2d$.
- Their projections on the camera sensor: $(u_1^{\text{raw}},\, v_1^{\text{raw}})$ and $(u_2^{\text{raw}},\, v_2^{\text{raw}})$.
- Camera intrinsics: image resolution (W, H), horizontal field of view θ_w, vertical field of view θ_h.

A.1.3 Reference frames

Definition 1 (Global frame $\mathcal{O}$) *Fixed, world-attached frame with axes (X, Y, Z). The beacons L_1, L_2 are given in this frame.*

Definition 2 (Remote frame $\mathcal{R}$) *Attached to the remote. Its origin coincides with the camera optical centre $\mathbf{P}$; its axes are the global axes rotated by the known Euler angles. All camera-model equations hold in this frame.*

A.1.4 Axis convention

The global frame $\mathcal{O}$ is oriented as follows: X runs horizontally along the reference bar (with L_1 on the positive side), Y is the vertical axis pointing upward, and Z is the depth axis pointing from the screen toward the remote. In particular, the screen plane is $Z = 0$ and the remote always sits at $Z > 0$.

A.2 Step 1 — Rotation Matrix from Euler Angles

A.2.1 Elementary rotation matrices

Three elementary rotations are defined about the Z, Y, and X axes respectively:

$$R_z(\psi) = \begin{pmatrix} \cos\psi & -\sin\psi & 0 \\ \sin\psi & \cos\psi & 0 \\ 0 & 0 & 1 \end{pmatrix}, \quad R_y(\theta) = \begin{pmatrix} \cos\theta & 0 & \sin\theta \\ 0 & 1 & 0 \\ -\sin\theta & 0 & \cos\theta \end{pmatrix}, \quad R_x(\phi) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos\phi & -\sin\phi \\ 0 & \sin\phi & \cos\phi \end{pmatrix}. \tag{3}$$

A.2.2 ZYX composition (intrinsic yaw–pitch–roll)

Using the ZYX intrinsic convention (first yaw about Z, then pitch about the new Y, then roll about the new X), the global-to-remote rotation matrix is:

$$\mathbf{R} = R_z(\psi)\, R_y(\theta)\, R_x(\phi). \tag{4}$$

A.2.3 Explicit expanded form

Defining the shorthand $c_\alpha \equiv \cos\alpha$, $s_\alpha \equiv \sin\alpha$:

$$\mathbf{R} = \begin{pmatrix} c_\psi c_\theta & c_\psi s_\theta s_\phi - s_\psi c_\phi & c_\psi s_\theta c_\phi + s_\psi s_\phi \\ s_\psi c_\theta & s_\psi s_\theta s_\phi + c_\psi c_\phi & s_\psi s_\theta c_\phi - c_\psi s_\phi \\ -s_\theta & c_\theta s_\phi & c_\theta c_\phi \end{pmatrix}. \tag{5}$$

A.2.4 Key property

Because $\mathbf{R}$ is orthogonal ($\mathbf{R}^\top \mathbf{R} = I_3$), its inverse equals its transpose:

$$\mathbf{R}^{-1} = \mathbf{R}^\top. \tag{6}$$

$\mathbf{R}$ transforms vectors from $\mathcal{O}$ to $\mathcal{R}$; $\mathbf{R}^\top$ transforms from $\mathcal{R}$ back to $\mathcal{O}$.

A.3 Step 2 — Camera Intrinsic Model

A.3.1 From FOV angles to pixel focal lengths

The pinhole camera geometry links the physical field of view to the focal length expressed in pixels. For the horizontal axis, half the image width subtends half the horizontal FOV:

$$\tan\left(\frac{\theta_w}{2}\right) = \frac{W/2}{f_x} \quad \Longrightarrow \quad \boxed{f_x = \frac{W}{2\tan(\theta_w/2)}}. \tag{7}$$

Likewise for the vertical axis:

$$\tan\left(\frac{\theta_h}{2}\right) = \frac{H/2}{f_y} \quad \Longrightarrow \quad \boxed{f_y = \frac{H}{2\tan(\theta_h/2)}}. \tag{8}$$

Remark 2 *If $\theta_w/W = \theta_h/H$ the pixels are square and $f_x = f_y$. Otherwise the camera has a non-unit pixel aspect ratio.*

A.3.2 Principal point

The optical axis intersects the sensor at its centre, called the *principal point*:

$$c_x = \frac{W}{2}, \qquad c_y = \frac{H}{2}. \tag{9}$$

A.3.3 Intrinsic matrix

The four intrinsic parameters are assembled into the 3×3 upper triangular matrix:

$$K = \begin{pmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix}. \tag{10}$$

A.3.4 Projection equation

A 3-D point $\mathbf{q} = (q_x, q_y, q_z)^\top$ in the remote frame $\mathcal{R}$ projects onto raw pixel coordinates via:

$$\begin{pmatrix} u^{\text{raw}} \\ v^{\text{raw}} \\ 1 \end{pmatrix} \sim K \begin{pmatrix} q_x/q_z \\ q_y/q_z \\ 1 \end{pmatrix}, \tag{11}$$

which gives explicitly:

$$u^{\text{raw}} = f_x \frac{q_x}{q_z} + c_x, \qquad v^{\text{raw}} = f_y \frac{q_y}{q_z} + c_y. \tag{12}$$

A.3.5 Back-projection: pixel to ray direction

Inverting (12), a raw pixel $(u^{\text{raw}}, v^{\text{raw}})$ defines a *ray direction* in $\mathcal{R}$. Since K is upper triangular its inverse is:

$$K^{-1} = \begin{pmatrix} 1/f_x & 0 & -c_x/f_x \\ 0 & 1/f_y & -c_y/f_y \\ 0 & 0 & 1 \end{pmatrix}. \tag{13}$$

The normalised (homogeneous) direction vector for observation i is:

$$\mathbf{d}_i = K^{-1} \begin{pmatrix} u_i^{\text{raw}} \\ v_i^{\text{raw}} \\ 1 \end{pmatrix} = \begin{pmatrix} (u_i^{\text{raw}} - c_x)/f_x \\ (v_i^{\text{raw}} - c_y)/f_y \\ 1 \end{pmatrix}. \tag{14}$$

The beacon L_i lies somewhere along the ray $\{ \lambda_i \, \mathbf{d}_i \mid \lambda_i > 0 \}$ in frame $\mathcal{R}$.

A.4 Step 3 — World-Space Ray Directions

A.4.1 Rotating the camera-frame ray into the global frame

The back-projection (14) gives the direction $\mathbf{d}_i$ of the ray towards beacon L_i expressed in the remote frame $\mathcal{R}$. Rotating it into the global frame $\mathcal{O}$ with $\mathbf{R}$ yields the *world-space ray direction*:

$$\mathbf{r}_i \;=\; \mathbf{R}\,\mathbf{d}_i \;\in\; \mathbb{R}^3. \tag{15}$$

$\mathbf{r}_i$ is fully known, depending only on the known rotation $\mathbf{R}$ and the observed pixel $\mathbf{d}_i$.

A.4.2 Ray constraint in the global frame

The beacon L_i lies somewhere along the ray emanating from the camera origin $\mathbf{P}$ in direction $\mathbf{r}_i$. Expressed entirely in $\mathcal{O}$:

$$L_i - \mathbf{P} = \lambda_i \, \mathbf{r}_i, \qquad \lambda_i > 0. \tag{16}$$

Rearranging, the position satisfies:

$$\boxed{\mathbf{P} = L_i - \lambda_i \, \mathbf{r}_i, \qquad i = 1, 2.} \tag{17}$$

A.5 Step 4 — Linear System for the Depths λ_1, λ_2

A.5.1 Equating the two position estimates

Equation (17) written for $i = 1$ and $i = 2$ must give the same $\mathbf{P}$:

$$L_1 - \lambda_1 \, \mathbf{r}_1 = L_2 - \lambda_2 \, \mathbf{r}_2. \tag{18}$$

Rearranging:

$$\lambda_1 \, \mathbf{r}_1 - \lambda_2 \, \mathbf{r}_2 = L_1 - L_2. \tag{19}$$

A.5.2 Right-hand side

With $L_1 = (d, 0, 0)^\top$ and $L_2 = (-d, 0, 0)^\top$:

$$L_1 - L_2 = \begin{pmatrix} 2d \\ 0 \\ 0 \end{pmatrix}. \tag{20}$$

A.5.3 Matrix form

Writing $\mathbf{r}_i = (r_x^i, \, r_y^i, \, r_z^i)^\top$, equation (19) becomes:

$$\underbrace{\begin{pmatrix} r_x^1 & -r_x^2 \\ r_y^1 & -r_y^2 \\ r_z^1 & -r_z^2 \end{pmatrix}}_{A \in \mathbb{R}^{3\times 2}} \begin{pmatrix} \lambda_1 \\ \lambda_2 \end{pmatrix} = \begin{pmatrix} 2d \\ 0 \\ 0 \end{pmatrix}. \tag{21}$$

A.5.4 Least-squares solution

System (21) is *overdetermined* (3 equations, 2 unknowns). The least-squares solution minimises $\|A\boldsymbol{\lambda} - \mathbf{b}\|^2$:

$$\begin{pmatrix}\lambda_1\\ \lambda_2\end{pmatrix} = (A^\top A)^{-1} A^\top \mathbf{b}, \qquad \mathbf{b} = \begin{pmatrix}2d\\ 0\\ 0\end{pmatrix}. \tag{22}$$

Explicit form of $A^\top A$

$$A^\top A = \begin{pmatrix}\|\mathbf{r}_1\|^2 & -\mathbf{r}_1\cdot\mathbf{r}_2\\ -\mathbf{r}_1\cdot\mathbf{r}_2 & \|\mathbf{r}_2\|^2\end{pmatrix}. \tag{23}$$

Its determinant is:

$$\det(A^\top A) = \|\mathbf{r}_1\|^2\|\mathbf{r}_2\|^2 - (\mathbf{r}_1\cdot\mathbf{r}_2)^2 = \|\mathbf{r}_1\times\mathbf{r}_2\|^2. \tag{24}$$

This vanishes if and only if $\mathbf{r}_1 \parallel \mathbf{r}_2$, i.e. the two beacons project onto the same pixel — a physically degenerate configuration.

Explicit form of $A^\top \mathbf{b}$

$$A^\top \mathbf{b} = \begin{pmatrix}2d\, r_x^1\\ -2d\, r_x^2\end{pmatrix}. \tag{25}$$

Closed-form solution Let $\alpha = \|\mathbf{r}_1\|^2$, $\beta = \mathbf{r}_1\cdot\mathbf{r}_2$, $\gamma = \|\mathbf{r}_2\|^2$, $\delta = \alpha\gamma - \beta^2$. Then:

$$\boxed{\lambda_1 = \frac{2d}{\delta}\big(\gamma\, r_x^1 - \beta\, r_x^2\big), \qquad \lambda_2 = \frac{2d}{\delta}\big(\beta\, r_x^1 - \alpha\, r_x^2\big).} \tag{26}$$

A.6 Step 5 — Recovering the 3-D Position P

A.6.1 Individual estimates

From (17):

$$\hat{\mathbf{P}}_1 = L_1 - \lambda_1\,\mathbf{r}_1 = \begin{pmatrix}d\\ 0\\ 0\end{pmatrix} - \lambda_1\,\mathbf{R}^\top\mathbf{d}_1, \tag{27}$$

$$\hat{\mathbf{P}}_2 = L_2 - \lambda_2\,\mathbf{r}_2 = \begin{pmatrix}-d\\ 0\\ 0\end{pmatrix} - \lambda_2\,\mathbf{R}^\top\mathbf{d}_2. \tag{28}$$

A.6.2 Final estimate

In the noiseless case $\hat{\mathbf{P}}_1 = \hat{\mathbf{P}}_2$. With noise, the average minimises the residual:

$$\boxed{\mathbf{P} = \frac{\hat{\mathbf{P}}_1 + \hat{\mathbf{P}}_2}{2}.} \tag{29}$$

A.7 Complete Algorithm

The full procedure is summarised below.

1. **Build the rotation matrix.** Compute $\mathbf{R} = R_z(\psi)\, R_y(\theta)\, R_x(\phi)$ using (5).
2. **Compute intrinsic parameters.**

$$f_x = \frac{W}{2\tan(\theta_w/2)}, \quad f_y = \frac{H}{2\tan(\theta_h/2)}, \quad c_x = \frac{W}{2}, \quad c_y = \frac{H}{2}.$$

3. **Back-project pixel observations.** For $i = 1, 2$, compute $\mathbf{d}_i$ via (14).
4. **Rotate rays in world frame.** $\mathbf{r}_i = \mathbf{R}\,\mathbf{d}_i$ (equation (15)).
5. **Solve for depths.** Assemble A from (21) and solve (22) to obtain λ_1, λ_2. The closed form is (26).
6. **Recover position.** $\mathbf{P} = \frac{1}{2}(L_1 - \lambda_1\mathbf{r}_1 + L_2 - \lambda_2\mathbf{r}_2)$.

A.8 Remarks and Edge Cases

Remark 3 (Degeneracy) *The system is degenerate when $\mathbf{r}_1 \parallel \mathbf{r}_2$ ($\delta = 0$). This happens only when the two beacons project onto identical pixels in the image, which is physically impossible if the bar is visible and the camera has finite resolution.*

Remark 4 (Sign of λ) *Both depths must be positive ($\lambda_i > 0$) for the beacons to be in front of the camera. A negative value indicates an inconsistent input (e.g. swapped beacon identities or an Euler-angle sign error).*